

目录

一、实验环境与实验内容	3
1.1 实验环境与硬件配置	3
1.2 实验内容	3
二、指令与数据通路说明	4
2.1 指令说明	4
2.2 整体数据通路	10
三、修改模块说明	11
3.1 顶层模块 openmips_min_sopc.v	11
3.2 控制器模块 ctrl.v	13
3.3 DMEM 模块 data_ram.v	14
3.4 七段数码管模块 seg7x16.v	15
3.5 约束文件修改	17
四、模块说明	18
4.1 PC 寄存器模块	18
4.2 IF 模块	18
4.3 ID 模块	19
4.4 EXE 模块	19
4.5 MEM 模块	20
4.6 WB 模块	20
五、仿真与下板验证	21
5.1 测试与调试方法	21
5.2 仿真验证与分析	22
5.3 下板验证	23
六、实验问题及解决方法	23
6.1 指令读取	23
6.2 顶层模块添加	24
6.3 debug 模块添加	24
6.4 中断起始地址修改	24
七、实验感想	24
参考文献	24

一、实验环境与实验内容

1.1 实验环境与硬件配置

处理器： 11th Gen Intel(R) Core(TM) i5-11300H @ 3.10GHz 3.11 GHz

开发平台：vivado 2019.2、ModelSim PE 10.4c

测试环境：MARS4.5

开发板： NEXYS 4 DDR Atrix-7

1.2 实验内容

1.2.1 实验目的

1. 回顾 MIPS 五阶段流水线 CPU 的基本结构
2. 学习教学用 CPU 各模块的编写以及模块之间的连接结构
3. 加深对 Verilog 语言的理解以及对硬件开发的理解
4. 为之后移植操作系统做准备工作

1.2.2 实验方案

参考《自己动手写 CPU》，改造 MIPS 流水线 CPU，使其支持 89 条指令。在《自己动手写 CPU》源码的基础上增加实现指令功能，完善总线及控制器等实现。本实验需要新增的 35 条指令具体如下所示：

1. 移动指令指令：movn, movz
2. 算术指令指令：clo, madd, maddu, msub, msubu
3. 跳转指令指令：b, bal, bgezal, bgtz, blez, bltz, bltzal
4. 加载存储指令指令：ll, lwl, lwr, sc, swl, swr
5. 异常相关指令指令：tge, tgeu, tlt, tltu, tne, teqi, tgei, tgeiu, tlti, tltiu, tnei
6. 其他指令：nop, ssnop, sync, pref

本实验中实现的 MIPS CPU 为五级流水线 CPU，分别为取指、译码、执行、访存、回写五个阶段，存储模式采取大端模式，整体采用哈佛结构，有独立的指令存储器和数据存储器。

二、指令与数据通路说明

2.1 指令说明

1. movn

1. 格式: `movn rd,rs,rt`

2. 描述: 如果 `rt` 不等于 0, 则 `rd` 获取 `rs` 的值。如果 `rt` 等于 0, 则 `rd` 的值不变, `movn` 指令的功能是在非零条件下移动寄存器的值。

3. 涉及的硬件: PC (程序计数器)、NPC (下一条指令的程序计数器)、IMEM (指令存储器)、RegFile (寄存器文件)。

2. movz

1. 格式: `movz rd,rs,rt`

2. 描述: 如果 `rt` 等于 0, 则 `rd` 获取 `rs` 的值。与 `movn` 指令相对, 其功能是在零条件下移动寄存器的值。

3. 涉及的硬件: PC、NPC、IMEM、RegFile。

3. clo

1. 格式: `clo rd,rs`

2. 描述: 该指令用于计算 `rs` 寄存器中最高位开始连续的“1”的个数。例如, 如果 `rs` 寄存器的值为 `0xFFFFFFFF`, 则 `clo` 指令会计算出 32, 因为该数值的二进制表示是连续 32 个“1”。

3. 涉及的硬件: PC、NPC、IMEM、RegFile。

4. madd

1. 格式: `madd rs,rt`

2. 描述: 该指令用于执行乘法加法操作, 它将 `rs` 和 `rt` 寄存器的值相乘, 然后将结果加到特殊寄存器 `HI,LO` 中。如果乘法结果超出了 32 位, `HI` 寄存器存储高 32 位, 而 `LO` 寄存器存储低 32 位。

3. 涉及的硬件: PC、NPC、IMEM、RegFile、HI,LO。

5. maddu

1. 格式: `maddu rs,rt`

2. 描述: 该指令类似于 `madd`, 但它用于无符号数的乘法加法操作。同样地, 乘法结果的高位存储在 `HI` 寄存器, 低位存储在 `LO` 寄存器。

3. 涉及的硬件: `PC`、`NPC`、`IMEM`、`RegFile`、`HI,LO`。

6. `msub`

1. 格式: `msub rs,rt`

2. 描述: 此指令将 `rs` 和 `rt` 寄存器的值相乘, 然后将结果从 `HI,LO` 寄存器中的现有值中减去。如果乘法结果超出了 32 位, 则 `HI` 寄存器存储高 32 位, 而 `LO` 寄存器存储低 32 位。

3. 涉及的硬件: `PC`、`NPC`、`IMEM`、`RegFile`、`HI,LO`。

7. `msubu`

1. 格式: `msubu rs,rt`

2. 描述: 当执行 `rs` 和 `rt` 的无符号数乘法操作, 结果会从特殊寄存器 `HI,LO` 中现有的值中减去。如果 `rs` 和 `rt` 的乘法结果超过了 32 位, 高位存储在 `HI` 寄存器, 低位存储在 `LO` 寄存器。`msubu` 指令的作用是执行无符号数乘法后的减法操作。

3. 涉及的硬件: `PC`、`NPC`、`IMEM`、`RegFile`、`HI,LO`。

8. `b`

1. 格式: `b offset`

2. 描述: 无条件分支, 当指令执行时, 处理器会跳转到 `b` 指令的当前位置加上 `offset` 的目标地址。`b` 指令简化了代码流程控制, 可以实现无条件的跳转。

3. 涉及的硬件: `PC`、`NPC`、`IMEM`、`RegFile`、`ALU`、`EXT18`、`ADD`。

9. `bal`

1. 格式: `bal offset`

2. 描述: 无条件分支, 同时将返回地址存储在寄存器 `$ra` 中, 方便函数调用后返回到原程序继续执行。`bal` 指令等效于 `bgezal` 指令的特殊情况, 即 `bgezal` 的条件总是满足的情况, 因此总是执行跳转。

3. 涉及的硬件: `PC`、`NPC`、`IMEM`、`RegFile`、`ALU`、`EXT18`、`ADD`。

10. `bgezal`

1. 格式: `bgezal rs,offset`

2. 描述：如果 `rs` 大于等于 0，则执行分支，并且将返回地址保存在寄存器 `$ra` 中，用于支持函数的调用与返回。该指令在实现函数调用时很有用，因为它提供了一个链接到返回地址的机制。

3. 涉及的硬件：PC、NPC、IMEM、RegFile、ALU、EXT18、ADD。

11. `bgtz`

1. 格式：`bgtz rs,offset`

2. 描述：如果 `rs` 大于 0，则执行分支操作。`bgtz` 指令用于条件分支，通常与循环或条件语句结合使用，以实现更复杂的控制流程。

3. 涉及的硬件：PC、NPC、IMEM、RegFile、ALU、EXT18、ADD。

12. `blez`

1. 格式：`blez rs,offset`

2. 描述：如果 `rs` 小于等于 0，则执行分支操作，处理器跳转到 `blez` 指令的当前位置加上 `offset` 的目标地址。`blez` 指令用于在 `rs` 寄存器的值不大于零时控制程序流程。

3. 涉及的硬件：PC、NPC、IMEM、RegFile、ALU、EXT18、ADD。

13. `bltz`

1. 格式：`bltz rs,offset`

2. 描述：如果 `rs` 小于 0，则执行分支操作，处理器跳转到 `bltz` 指令的当前位置加上 `offset` 的目标地址。`bltz` 指令专用于在 `rs` 寄存器的值为负数时实现程序的条件分支。

3. 涉及的硬件：PC、NPC、IMEM、RegFile、ALU、EXT18、ADD。

14. `bltzal`

1. 格式：`bltzal rs,offset`

2. 描述：如果 `rs` 小于 0，则执行分支，并且将返回地址保存在寄存器 `$ra` 中。`bltzal` 指令不仅用于条件分支，而且用于支持函数调用与返回，使得程序能够在执行完函数后返回到原来的位置。

3. 涉及的硬件：PC、NPC、IMEM、RegFile、ALU、EXT18、ADD。

15. `ll`

1. 格式：`ll rt, offset(base)`

2. 描述: ll 指令用于从内存中加载一个字到寄存器 rt, 同时设置内存系统的一个监测位, 用于后续的 sc (Store Conditional) 指令实现原子操作。如果在执行 sc 指令之前该地址被修改, sc 操作将失败。

3. 涉及的硬件: PC、NPC、IMEM、RegFile、ALU、DMEM。

16. lwl

1. 格式: lwl rt, offset(base)

2. 描述: 该指令用于加载字的左边部分到寄存器 rt, 一般与 lwr 指令组合使用来加载非对齐的字。lwl 指令根据对齐的字的边界, 从内存中读取字的左边部分, 拼接到寄存器 rt 中。与 lhu、lh 指令的作用不同, 后两者是加载半字。

3. 涉及的硬件: PC、NPC、IMEM、RegFile、ALU、EXT16、DMEM。

17. lwr

1. 格式: lwr rt, offset(base)

2. 描述: 与 lwl 指令相对应, 用于加载字的右边部分到寄存器 rt, 一般与 lwl 指令组合使用来加载非对齐的字。

3. 涉及的硬件: PC、NPC、IMEM、RegFile、ALU、EXT16、DMEM。

18. sc

1. 格式: sc rt, offset(base)

2. 描述: 该指令用于实现原子交换操作, 也就是 LLbit (Load Linked bit) 为 1 时, 将寄存器 rt 的内容存储到指定的内存地址, 并将 LLbit 设为 0。如果 RMW (Read Modify Write) 指令组执行了此操作, 也就是 LLbit 为 0, 则不会执行存储, 并将寄存器 rt 中的内容设为 0。

3. 涉及的硬件: PC、NPC、IMEM、RegFile、ALU、EXT16、DMEM。

19. swl

1. 格式: swl rt, offset(base)

2. 描述: 将寄存器 rt 中的字的左边部分存储到内存的指定地址中, 通常与 swr 指令配合使用以实现非对齐的字的存储操作。swl 指令根据对齐的字边界, 将寄存器 rt 中的左边部分存储到内存。与 sh 指令的作用不同, 后者是存储半字。

3. 涉及的硬件: PC、NPC、IMEM、RegFile、ALU、EXT16、DMEM。

20. swr

1. 格式: `swr rt, offset(base)`
2. 描述: 将寄存器 `rt` 中的字的右边部分存储到内存的指定地址中, 通常与 `swl` 指令配合使用以实现非对齐的字的存储操作。
3. 涉及的硬件: PC、NPC、IMEM、RegFile、ALU、EXT16、DMEM。

21. `tge`

1. 格式: `tge rs, rt`
2. 描述: 如果通用寄存器 `rs` 的值大于等于寄存器 `rt` 的值, 则执行陷阱 (`trap`) 操作。`tge` 指令用于在寄存器的值满足特定的条件时产生陷阱, 用于程序的异常处理或调试。
3. 涉及的硬件: PC、NPC、CPO、IMEM、RegFiles、ALU。

22. `tgeu`

1. 格式: `tgeu rs, rt`
2. 描述: 如果通用寄存器 `rs` 的值在无符号比较下大于等于寄存器 `rt` 的值, 则执行陷阱操作。`tgeu` 与 `tge` 指令相似, 但专用于无符号数的比较。
3. 涉及的硬件: PC、NPC、CPO、IMEM、RegFiles、ALU。

23. `tlt`

1. 格式: `tlt rs, rt`
2. 描述: 如果通用寄存器 `rs` 的值小于寄存器 `rt` 的值, 则执行陷阱操作。`tlt` 指令用于在寄存器的值不满足特定的条件时产生陷阱, 用于程序的异常处理或调试。
3. 涉及的硬件: PC、NPC、CPO、IMEM、RegFiles、ALU。

24. `tltu`

1. 格式: `tltu rs, rt`
2. 描述: 如果通用寄存器 `rs` 的值在无符号比较下小于寄存器 `rt` 的值, 则执行陷阱操作。`tltu` 指令用于在寄存器的值不满足特定的无符号比较条件时产生陷阱, 用于程序的异常处理或调试。
3. 涉及的硬件: PC、NPC、CPO、IMEM、RegFiles、ALU。

25. `tne`

1. 格式: `tne rs, rt`

2. 描述：如果通用寄存器 rs 的值不等于寄存器 rt 的值，则执行陷阱操作。 tne 指令用于在寄存器的值不满足等值条件时产生陷阱，用于程序的异常处理或调试。

3. 涉及的硬件：PC、NPC、CPO、IMEM、RegFiles、ALU。

26. $teqi$

1. 格式： $teqi\ rs, immediate$

2. 描述：如果通用寄存器 rs 的值等于立即数（立即数被符号扩展到 32 位）则执行陷阱操作。 $teqi$ 指令用于比较寄存器值和立即数，如果相等，则产生陷阱。

3. 涉及的硬件：PC、NPC、CPO、IMEM、RegFiles、ALU。

27. $tgei$

1. 格式： $tgei\ rs, immediate$

2. 描述：如果通用寄存器 rs 的值大于等于立即数（立即数被符号扩展到 32 位）则执行陷阱操作。 $tgei$ 指令用于比较寄存器值和立即数，如果寄存器值大等于立即数，则产生陷阱。

3. 涉及的硬件：PC、NPC、CPO、IMEM、RegFiles、ALU。

28. $tgeiu$

1. 格式： $tgeiu\ rs, immediate$

2. 描述：如果通用寄存器 rs 的值在无符号比较下大于或等于立即数 $immediate$ ，则执行陷阱操作。 $tgeiu$ 指令用于在寄存器的值无符号比较满足或超过特定立即数时产生陷阱，通常用于程序的异常处理或调试。

3. 涉及的硬件：PC、NPC、CPO、IMEM、RegFiles、ALU。

29. $tlti$

1. 格式： $tlti\ rs, immediate$

2. 描述：如果通用寄存器 rs 的值小于立即数 $immediate$ ，则执行陷阱操作。 $tlti$ 指令用于在寄存器的值小于特定立即数时产生陷阱，用于程序的异常处理或调试。

3. 涉及的硬件：PC、NPC、CPO、IMEM、RegFiles、ALU。

30. $tltiu$

1. 格式： $tltiu\ rs, immediate$

2. 描述：如果通用寄存器 `rs` 的值在无符号比较下小于立即数 `immediate`，则执行陷阱操作。`tltiu` 指令用于在寄存器的值无符号比较小于特定立即数时产生陷阱，用于程序的异常处理或调试。

3. 涉及的硬件：PC、NPC、CPO、IMEM、RegFiles、ALU。

31. `tnei`

1. 格式：`tnei rs, immediate`

2. 描述：如果通用寄存器 `rs` 的值不等于立即数 `immediate`，则执行陷阱操作。`tnei` 指令用于在寄存器的值不等于特定立即数时产生陷阱，用于程序的异常处理或调试。

3. 涉及的硬件：PC、NPC、CPO、IMEM、RegFiles、ALU。

32. `nop`

1. 描述：无操作。

33. `ssnop`

1. 描述：一个特殊类型的空操作，此类指令在某些 MIPS CPU 中的 `nop` 指令的使用上有特殊的效果，可以防止与 `nop` 指令相关的潜在执行速度降低的问题。

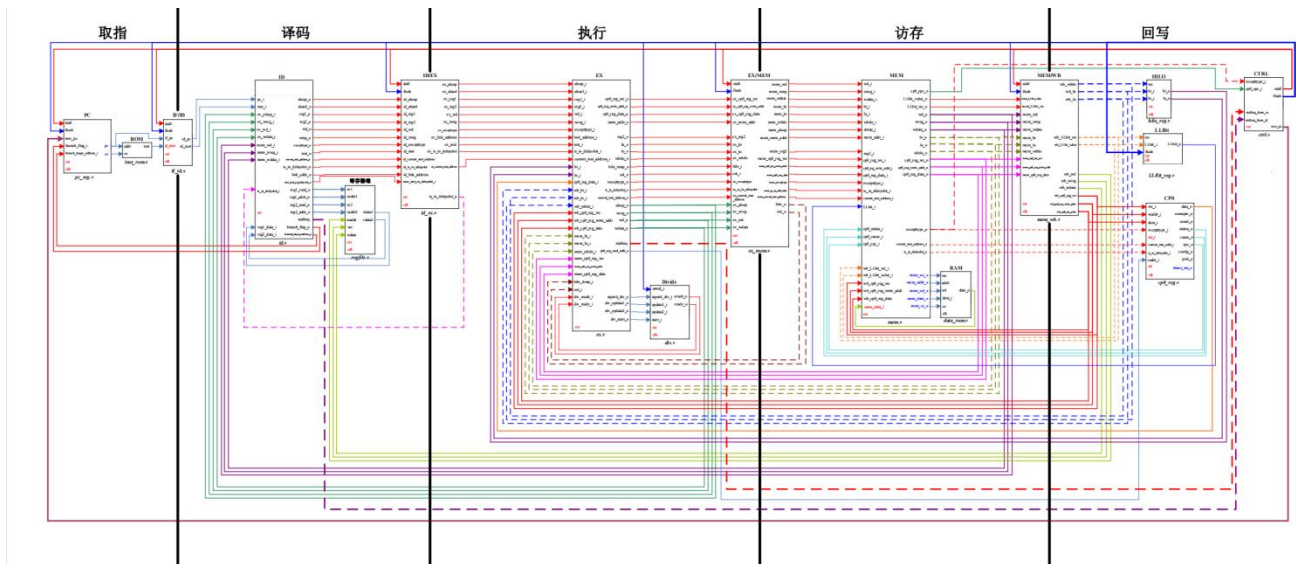
34. `sync`

1. 描述：用于保证内存操作的完成，以确保在此指令之前的所有内存操作都已经正确完成，以确保数据的一致性。在多核心或多线程环境中，`sync` 指令用于在不同的处理器或线程之间同步内存操作。

35. `pref`

1. 描述：用于数据预取操作，此指令在 MIPS CPU 中用于预先将数据加载到缓存中，可以提升后续访问数据的速度。`pref` 指令可视为 `nop` 指令的增强版。

2.2 整体数据通路



三、修改模块说明

3.1 顶层模块 openmips_min_soc.v

本模块需要将 CPU 模块、指令存储器、数据存储器、七段数码管控制器实例化，并完成个模块之间的连接。同时顶层模块中需要实现分频器，保证数码管的显示变化可以被人眼捕捉。具体实现为：

```

1.  `include "defines.v"
2.
3.  module openmips_min_soc(
4.      input wire clk,
5.      input wire rst,
6.      output [7:0] O_Seg,
7.      output [7:0] O_Sel
8.  );
9.
10.     reg clk_s;
11.     initial clk_s=0;
12.     integer cnt=0;
13.     always @(posedge clk)
14.     begin
15.
16.     if(cnt<1)
17.         cnt=cnt+1;
18.     else
19.         begin
20.             cnt=0;
21.             clk_s=~clk_s;
22.         end
23.     end
24.
25.
26.     //连接指令存储器
    
```

```
27. wire[`InstAddrBus] inst_addr;
28. wire[`InstBus] inst;
29.
30. wire rom_ce;
31. wire mem_we_i;
32. wire[`RegBus] mem_addr_i;
33. wire[`RegBus] mem_data_i;
34. wire[`RegBus] mem_data_o;
35. wire[3:0] mem_sel_i;
36. wire mem_ce_i;
37. wire[5:0] int;
38. wire timer_int;
39. wire[`DataBus] Seg7_In;
40.
41. //assign int = {5'b00000, timer_int, gpio_int, uart_int};
42. assign int = {5'b00000, timer_int};
43.
44. openmips openmips0(
45.     .clk(clk_s),
46.     .rst(rst),
47.     .rom_addr_o(inst_addr),
48.     .rom_data_i(inst),
49.     .rom_ce_o(rom_ce),
50.     .int_i(int),
51.     .ram_we_o(mem_we_i),
52.     .ram_addr_o(mem_addr_i),
53.     .ram_sel_o(mem_sel_i),
54.     .ram_data_o(mem_data_i),
55.     .ram_data_i(mem_data_o),
56.     .ram_ce_o(mem_ce_i),
57.     .timer_int_o(timer_int)
58. );
59.
60. inst_rom inst_rom0(
61.     .ce(rom_ce),
62.     .addr(inst_addr),
63.     .inst(inst)
64. );
65.
66. data_ram data_ram0(
67.     .clk(clk_s),
68.     .ce(mem_ce_i),
69.     .we(mem_we_i),
70.     .addr(mem_addr_i),
71.     .sel(mem_sel_i),
72.     .data_i(mem_data_i),
73.     .data_o(mem_data_o),
74.     .debugreg(Seg7_In)
75. );
76.
77. Seg7x16 Seg7(clk, rst, 1, Seg7_In, 0_Seg, 0_Sel);
```

78. `endmodule`

3.2 控制器模块 `ctrl.v`

中断例程起始地址为 `0x00400004`，需要修改 `ctrl.v` 中的跳转地址，将中断跳转地址修改为 `0x00400004`，从而保证最后输出结果符合预期。具体实现为：

```

1.  `include "defines.v"
2.
3.  module ctrl(
4.      input wire rst,
5.      input wire[31:0] excepttype_i,
6.      input wire[`RegBus] cp0_epc_i,
7.      input wire stallreq_from_id,
8.      //来自执行阶段的暂停请求
9.      input wire stallreq_from_ex,
10.     output reg[`RegBus] new_pc,
11.     output reg flush,
12.     output reg[5:0] stall
13. );
14.     always @ (*) begin
15.         if(rst == `RstEnable) begin
16.             stall <= 6'b000000;
17.             flush <= 1'b0;
18.             new_pc <= `ZeroWord;
19.         end else if(excepttype_i != `ZeroWord) begin
20.             flush <= 1'b1;
21.             stall <= 6'b000000;
22.             case (excepttype_i)
23.                 32'h00000001: begin //interrupt
24.                     new_pc <= 32'h00000020;
25.                 end
26.                 32'h00000008: begin //syscall
27.                     new_pc <= 32'h00400004;
28.                 end
29.                 32'h0000000a: begin //inst_invalid
30.                     new_pc <= 32'h00400004;
31.                 end
32.                 32'h0000000d: begin //trap
33.                     new_pc <= 32'h00400004;
34.                 end
35.                 32'h0000000c: begin //ov
36.                     new_pc <= 32'h00400004;
37.                 end
38.                 32'h0000000e: begin //eret
39.                     new_pc <= cp0_epc_i;
40.                 end
41.                 default : begin
42.                     end
43.             endcase

```

```

44.     end else if(stallreq_from_ex == `Stop) begin
45.         stall <= 6'b001111;
46.         flush <= 1'b0;
47.     end else if(stallreq_from_id == `Stop) begin
48.         stall <= 6'b000111;
49.         flush <= 1'b0;
50.     end else begin
51.         stall <= 6'b000000;
52.         flush <= 1'b0;
53.         new_pc <= `ZeroWord;
54.     end //if
55. end //always
56. endmodule

```

3.3 DMEM 模块 data_ram.v

在 DMEM 中，需要减去起始地址 0x00400000 进行地址变换

```

1.  `include "defines.v"
2.
3.  module data_ram(
4.      input wire      clk,
5.      input wire      ce,
6.      input wire      we,
7.      input wire[`DataAddrBus] addr,
8.      input wire[3:0] sel,
9.      input wire[`DataBus] data_i,
10.     output reg[`DataBus] data_o,
11.     output[`DataBus] debugreg
12. );
13.
14.     reg[`ByteWidth] data_mem0[0:`DataMemNum-1];
15.     reg[`ByteWidth] data_mem1[0:`DataMemNum-1];
16.     reg[`ByteWidth] data_mem2[0:`DataMemNum-1];
17.     reg[`ByteWidth] data_mem3[0:`DataMemNum-1];
18.
19.
20.     always @ (posedge clk) begin
21.         if (ce == `ChipDisable) begin
22.             data_o <= `ZeroWord;
23.         end else if(we == `WriteEnable) begin
24.             if (sel[3] == 1'b1) begin
25.                 data_mem3[addr[`DataMemNumLog2+1:2]-
17' b0_0100_0000_0000_0000] <= data_i[31:24];
26.             end
27.             if (sel[2] == 1'b1) begin
28.                 data_mem2[addr[`DataMemNumLog2+1:2]-
17' b0_0100_0000_0000_0000] <= data_i[23:16];
29.             end

```

```

30.         if (sel[1] == 1'b1) begin
31.             data_mem1[addr[`DataMemNumLog2+1:2]-
17'b0_0100_0000_0000_0000] <= data_i[15:8];
32.         end
33.         if (sel[0] == 1'b1) begin
34.             data_mem0[addr[`DataMemNumLog2+1:2]-
17'b0_0100_0000_0000_0000] <= data_i[7:0];
35.         end
36.     end
37. end
38.
39. always @ (*) begin
40.     if (ce == `ChipDisable) begin
41.         data_o <= `ZeroWord;
42.     end else if (we == `WriteDisable) begin
43.         data_o <= {data_mem3[addr[`DataMemNumLog2+1:2]-17'b0_0100_0000_0000_0000],
44.                     data_mem2[addr[`DataMemNumLog2+1:2]-17'b0_0100_0000_0000_0000],
45.                     data_mem1[addr[`DataMemNumLog2+1:2]-17'b0_0100_0000_0000_0000],
46.                     data_mem0[addr[`DataMemNumLog2+1:2]-17'b0_0100_0000_0000_0000]};
47.     end else begin
48.         data_o <= `ZeroWord;
49.     end
50. end
51.
52.     assign debugreg={data_mem3[17'b0_0000_0000_0000_0000],
53.                       data_mem2[17'b0_0000_0000_0000_0000],
54.                       data_mem1[17'b0_0000_0000_0000_0000],
55.                       data_mem0[17'b0_0000_0000_0000_0000]};
56. endmodule

```

3.4 七段数码管模块 seg7x16.v

将指定的 0x10010000 内存单元最终结果显示在数码管上，实现沿用 MIPS54 的气短数码管驱动模块。具体实现为：

```

1.  `timescale 1ns / 1ns
2.
3.  module Seg7x16(
4.      input Clk,
5.      input Reset,
6.      input Cs,
7.      input [31:0] I_Data,
8.      output [7:0] O_Seg,
9.      output [7:0] O_Sel
10.  );
11.

```

```
12.     reg [14:0] Cnt;
13.     always @ (posedge Clk, posedge Reset)
14.         if (Reset)
15.             Cnt <= 0;
16.         else
17.             Cnt <= Cnt + 1'B1;
18.
19.     wire Seg7_Clk = Cnt[14];
20.
21.     reg [2:0] Seg7_Addr;
22.
23.     always @ (posedge Seg7_Clk, posedge Reset)
24.         if(Reset)
25.             Seg7_Addr <= 0;
26.         else
27.             Seg7_Addr <= Seg7_Addr + 1'B1;
28.
29.     reg [7:0] O_Sel_R;
30.
31.     always @ (*)
32.         case(Seg7_Addr)
33.             7 : O_Sel_R = 8'B01111111;
34.             6 : O_Sel_R = 8'B10111111;
35.             5 : O_Sel_R = 8'B11011111;
36.             4 : O_Sel_R = 8'B11101111;
37.             3 : O_Sel_R = 8'B11110111;
38.             2 : O_Sel_R = 8'B11111011;
39.             1 : O_Sel_R = 8'B11111101;
40.             0 : O_Sel_R = 8'B11111110;
41.         endcase
42.
43.     reg [31:0] I_Data_Store;
44.     always @ (posedge Clk, posedge Reset)
45.         if(Reset)
46.             I_Data_Store <= 0;
47.         else if(Cs)
48.             I_Data_Store <= I_Data;
49.
50.     reg [7:0] Seg_Data_R;
51.     always @ (*)
52.         case(Seg7_Addr)
53.             0 : Seg_Data_R = I_Data_Store[3:0];
54.             1 : Seg_Data_R = I_Data_Store[7:4];
55.             2 : Seg_Data_R = I_Data_Store[11:8];
56.             3 : Seg_Data_R = I_Data_Store[15:12];
57.             4 : Seg_Data_R = I_Data_Store[19:16];
58.             5 : Seg_Data_R = I_Data_Store[23:20];
59.             6 : Seg_Data_R = I_Data_Store[27:24];
60.             7 : Seg_Data_R = I_Data_Store[31:28];
61.         endcase
62.
```

```
63.     reg [7:0] O_Seg_R;
64.     always @ (posedge Clk, posedge Reset)
65.     if(Reset)
66.         O_Seg_R <= 8'Hff;
67.     else
68.         case(Seg_Data_R)
69.             4'H0 : O_Seg_R <= 8'HC0;
70.             4'H1 : O_Seg_R <= 8'HF9;
71.             4'H2 : O_Seg_R <= 8'HA4;
72.             4'H3 : O_Seg_R <= 8'HB0;
73.             4'H4 : O_Seg_R <= 8'H99;
74.             4'H5 : O_Seg_R <= 8'H92;
75.             4'H6 : O_Seg_R <= 8'H82;
76.             4'H7 : O_Seg_R <= 8'HF8;
77.             4'H8 : O_Seg_R <= 8'H80;
78.             4'H9 : O_Seg_R <= 8'H90;
79.             4'HA : O_Seg_R <= 8'H88;
80.             4'HB : O_Seg_R <= 8'H83;
81.             4'HC : O_Seg_R <= 8'HC6;
82.             4'HD : O_Seg_R <= 8'HA1;
83.             4'HE : O_Seg_R <= 8'H86;
84.             4'HF : O_Seg_R <= 8'H8E;
85.         endcase
86.
87.     assign O_Sel = O_Sel_R;
88.     assign O_Seg = O_Seg_R;
89.
90. endmodule
```

3.5 约束文件修改

约束文件主要修改选择信号、时钟信号、复位信号引脚，具体实现修改为

```
1.     set_property PACKAGE_PIN E3 [get_ports clk]
2.     set_property PACKAGE_PIN N17 [get_ports rst]
3.
4.     set_property PACKAGE_PIN T10 [get_ports {O_Seg[0]}]
5.     set_property PACKAGE_PIN R10 [get_ports {O_Seg[1]}]
6.     set_property PACKAGE_PIN K16 [get_ports {O_Seg[2]}]
7.     set_property PACKAGE_PIN K13 [get_ports {O_Seg[3]}]
8.     set_property PACKAGE_PIN P15 [get_ports {O_Seg[4]}]
9.     set_property PACKAGE_PIN T11 [get_ports {O_Seg[5]}]
10.    set_property PACKAGE_PIN L18 [get_ports {O_Seg[6]}]
11.    set_property PACKAGE_PIN H15 [get_ports {O_Seg[7]}]
12.
13.    set_property PACKAGE_PIN J17 [get_ports {O_Sel[0]}]
14.    set_property PACKAGE_PIN J18 [get_ports {O_Sel[1]}]
15.    set_property PACKAGE_PIN T9 [get_ports {O_Sel[2]}]
16.    set_property PACKAGE_PIN J14 [get_ports {O_Sel[3]}]
17.    set_property PACKAGE_PIN P14 [get_ports {O_Sel[4]}]
```



```

18.  set_property PACKAGE_PIN T14 [get_ports {0_Sel[5]}]
19.  set_property PACKAGE_PIN K2 [get_ports {0_Sel[6]}]
20.  set_property PACKAGE_PIN U13 [get_ports {0_Sel[7]}]
21.
22.  set_property IOSTANDARD LVCMOS33 [get_ports {0_Sel[7]}]
23.  set_property IOSTANDARD LVCMOS33 [get_ports {0_Sel[6]}]
24.  set_property IOSTANDARD LVCMOS33 [get_ports {0_Sel[5]}]
25.  set_property IOSTANDARD LVCMOS33 [get_ports {0_Sel[4]}]
26.  set_property IOSTANDARD LVCMOS33 [get_ports {0_Sel[3]}]
27.  set_property IOSTANDARD LVCMOS33 [get_ports {0_Sel[2]}]
28.  set_property IOSTANDARD LVCMOS33 [get_ports {0_Sel[1]}]
29.  set_property IOSTANDARD LVCMOS33 [get_ports {0_Sel[0]}]
30.
31.  set_property IOSTANDARD LVCMOS33 [get_ports {0_Seg[7]}]
32.  set_property IOSTANDARD LVCMOS33 [get_ports {0_Seg[6]}]
33.  set_property IOSTANDARD LVCMOS33 [get_ports {0_Seg[5]}]
34.  set_property IOSTANDARD LVCMOS33 [get_ports {0_Seg[4]}]
35.  set_property IOSTANDARD LVCMOS33 [get_ports {0_Seg[3]}]
36.  set_property IOSTANDARD LVCMOS33 [get_ports {0_Seg[2]}]
37.  set_property IOSTANDARD LVCMOS33 [get_ports {0_Seg[1]}]
38.  set_property IOSTANDARD LVCMOS33 [get_ports {0_Seg[0]}]
39.
40.  set_property IOSTANDARD LVCMOS33 [get_ports rst]
41.  set_property IOSTANDARD LVCMOS33 [get_ports clk]
42.
43.  create_clock -period 200.000 -name clk_pin -waveform {0.000 100.000} [get_ports clk]
44.  set_input_delay -clock [get_clocks *] 1.000 [get_ports rst]
45.  set_output_delay -clock [get_clocks *] 0.000 [get_ports -
filter { NAME =~ "*" && DIRECTION == "OUT" }]

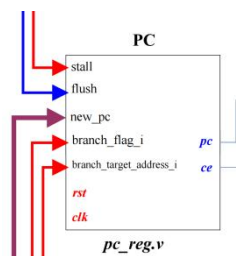
```

四、模块说明

4.1 PC 寄存器模块

功能：存储当前指令的地址，并根据控制信号和指令流水线的状态更新 PC 寄存器的值。

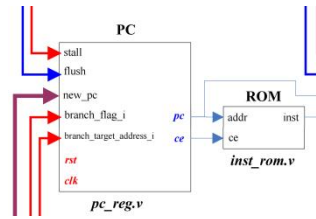
模块设计：



4.2 IF 模块

功能：取指模块，主要包含指令寄存器等。

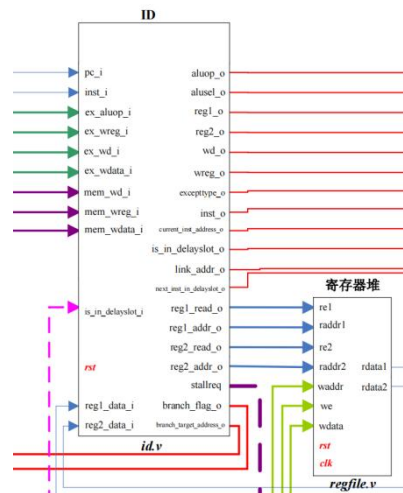
模块设计：



4.3 ID 模块

功能：主要包含控制单元模块、通用寄存器堆等

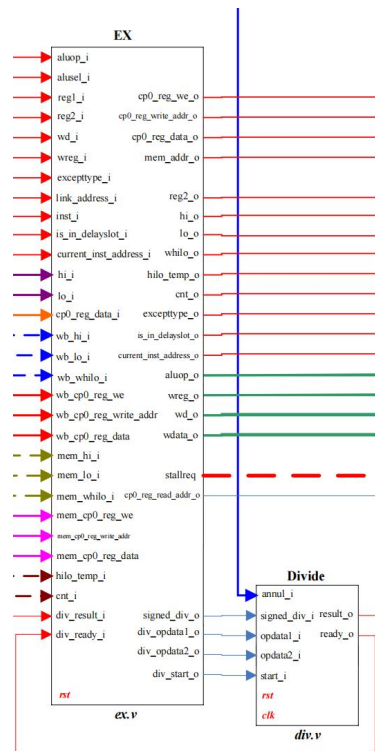
模块设计：



4.4 EXE 模块

功能：执行模块，包含除法器模块。

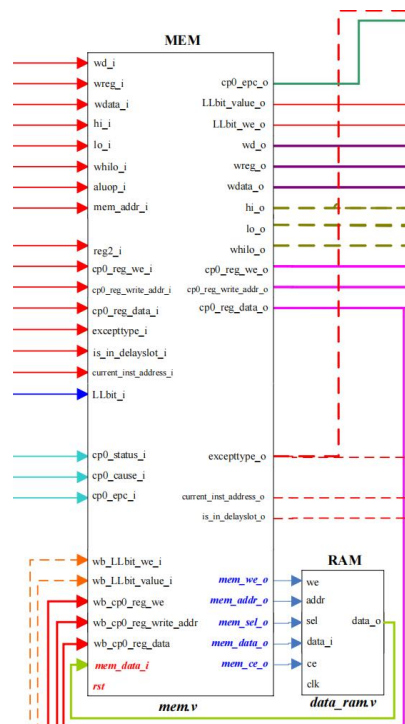
模块设计：



4.5 MEM 模块

功能：访存模块，主要包含数据存储器等模块

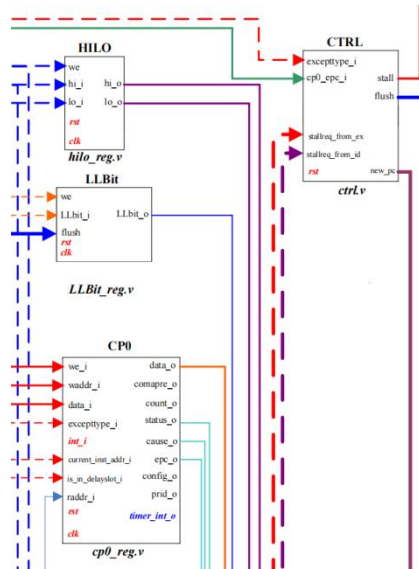
模块设计：



4.6 WB 模块

功能：回写模块，主要包含 HI 寄存器、LO 寄存器、CP0 寄存器等模块。

模块设计：



五、仿真与下板验证

5.1 测试与调试方法

本次实验采用 Vivado 2019.2 自带仿真工具进行前后仿真，观察波形变化。采用的测试 testbench 文件实现如下：

```

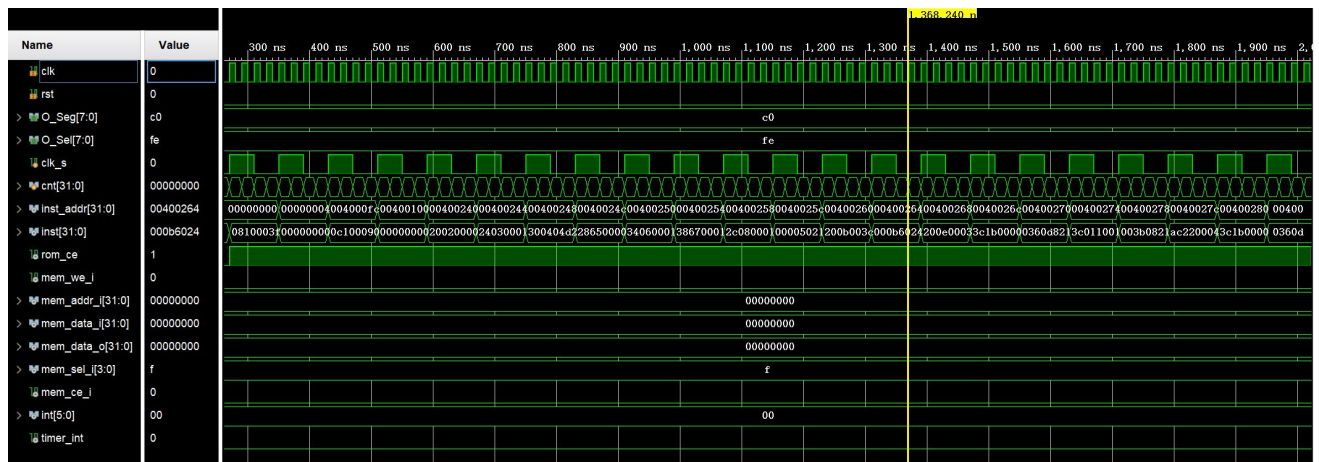
1.  `include "defines.v"
2.  `timescale 1ns/1ps
3.
4.  module openmips_min_sopc_tb();
5.
6.      reg    CLOCK_50;
7.      reg    rst;
8.
9.
10.     initial begin
11.         CLOCK_50 = 1'b0;
12.         forever #10 CLOCK_50 = ~CLOCK_50;
13.     end
14.
15.     initial begin
16.         rst = `RstEnable;
17.     end
18.
19.     openmips_min_sopc openmips_min_sopc0(
20.         .clk(CLOCK_50),
21.         .rst(rst)
22.     );
23.
24. endmodule

```

5.2 仿真验证与分析

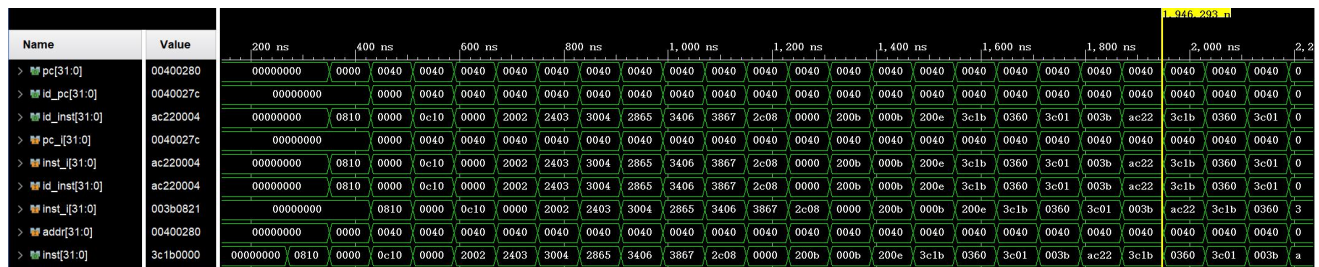
测试指令由老师给出的汇编文件 测试程序 (mips_89_mars_board_big).s 经过 Mars 进行反汇编生成 COE 文件，注意此处直接将 COE 文件视作文本文件，在 inst_rom.v 中指定文本文件绝对路径，将指令读取出来之后存储于 inst_mem 当中，在测试时将指令逐条取出即可。以下进行前仿真测试。

首先观察指令读取情况



可以观察到指令可以正常读取，指令地址一次递增，取指正常

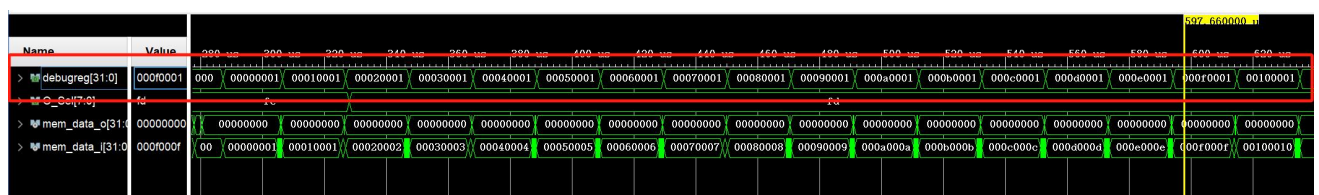
观察流经各个部件的指令和 pc 相关信号



可以观察到流水线能够正常流动

在 data_ram0 中添加 debug_reg 接口，将高四位赋值为时钟中断计数，将后四位赋值为错误码，在顶层模块中将 debug_reg 与七段数码管连接即可，进行仿真观察波形

查看 debug_reg



可以观察到 debug_reg 前四位随着时钟中断计数不断增加，后四位保持不变，为 0x0001

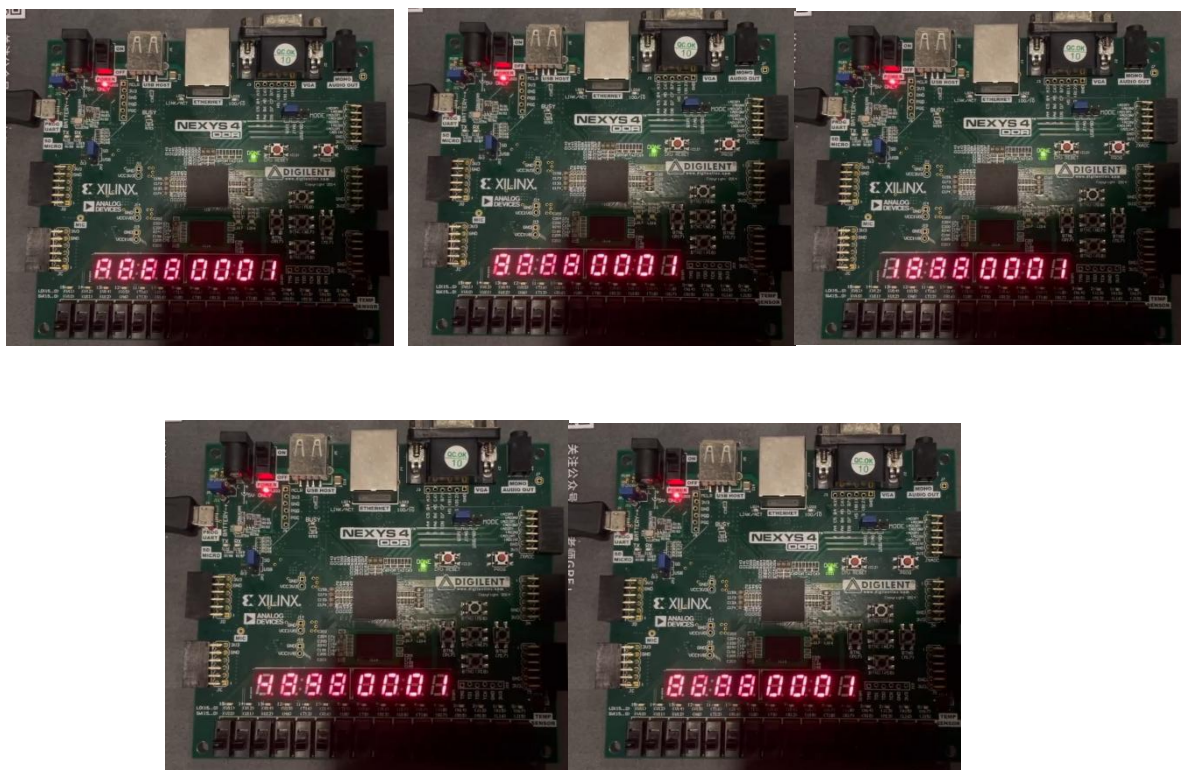
debug 寄存器的赋值部分为:

1.	assign debugreg={data_mem3[17'b0_0000_0000_0000_0000],
2.	data_mem2[17'b0_0000_0000_0000_0000],
3.	data_mem1[17'b0_0000_0000_0000_0000],
4.	data_mem0[17'b0_0000_0000_0000_0000]};

波形图展示与 debug_reg 寄存器变化结果一致, 数码管低 半字显示 0x0001, 高半字随着时钟中断而计数, 与预期一致。

5.3 下板验证

如图生成 bit 并下板后, 高半字随着时钟中断计数, 低半字显示 0x0001, 测试符合预期。



六、实验问题及解决方法

6.1 指令读取

给出的测试文件为汇编文件, 需要首先通过反汇编工具 (此处选择 MARS) 将其转换为二进制指令。本 CPU 设计时采用直接从文本文件中读取二进制代码, 预先存储于数组中后逐条进行读取。此方法相较于 COE+IP 核的方法, 虽然对内存 (寄存器) 消耗较大, 但相对于多次 I/O 而言节省了部分时间。详细实现在 inst_ram.v 中。

6.2 顶层模块添加

由于《自己动手实现 CPU》中代码缺少顶层模块，我们需要手动补充顶层模块，从而完成后续的仿真和下板验证任务。同时需要在顶层模块中引入分频器，将原时钟频率进行 $1e6$ 倍分频，保证在下板时七段数码管的变化能够让人眼捕捉到。

6.3 debug 模块添加

在实验测试文件说明中，给出了测试要求：正确结果，数码管低半字显示 0x0001，高半字随着时钟中断而计数；错误结果，数码管显示-1~-10 的错误码，或者数码管低半字显示 0x0001 但高半字不计数。则需要在 data_ram.v 中添加 debug reg 引脚，将对应的数据切片整合到 debug reg 数据中，在顶层模块中连接好七段数码管进行显示。

6.4 中断起始地址修改

在中断处理模块中(cp.v) 需要更改起始地址，在测试文件说明中给出了起始单元 0x00400000，与源代码中 0x00000040 不同。

七、实验感想

通过该实验，我的对 CPU 工作流程的理解进一步加深。此次实验的主要任务是修改《自己动手写 CPU》雷思磊书中的 CPU 设计，使其能够在我们的 Nexys 4 开发板上顺利运行。在进行改造之前需要对源代码有充分理解，需要尤其注意模块连接关系代码实现细节、CPU 工作流程逻辑等。在调整任何单一模块时，必须全面考虑其对整个系统的影响，注意是都需要修改控制信号等。

在本次实验中，我也丰富了我的 Debug 技巧，在 Vivado 的波形图界面选择 Scope -> Objects 的模块中的不同信号进行检测。同时在测试过程中我注意到了仿真时间的影响。在进行 debug_ram 信号观察时，由于仿真时间过短，导致我错误判断 CPU 连接存在问题，在信号连接上浪费了部分时间，最后通过调整 Vivado 模拟器的模拟时间解决问题。

本实验为后续操作系统移植实验的基础，希望在后续实验中，我能够对本实验完成的支持 89 条指令的 CPU 进行进一步完善。

参考文献

- [1] 秦国锋, 王力生, 陆有军, 郭玉臣. 计算机系统结构实验指导, 清华大学出版社, 2019.
- [2] 雷思磊. 自己动手写 CPU, 电子工业出版社, 2014.
- [3] 张晨曦, 王志英等. 计算机体系结构, 高等教育出版社, 2014.